SC4001 Neural Networks and Deep Learning Assignment: Part (b)

Aaron Soh
College of Computing and Data Science
Nanyang Technological University
Singapore
asoh014@e.ntu.edu.sg

Abstract

In part (b) of the assignment, we perform classification with convolutional neural network (CNN) models on a dataset of chest X-ray images, to identify pneumonia patients from healthy subjects. We train a total of 3 models (AlexNet, VGG, ResNet) with varying hyperparameters and evaluate their performance in this report.

1 Generative AI Declaration

In this assignment, I used ChatGPT to assist with the implementation of GPU support for training our models, by moving the input tensors and model to the GPU, speeding up training and hyperparameter search. This involved patching my model architecture code with various calls to move the tensors to GPU, and detaching them later on for data plotting on CPU. All AI-generated bug fixes was carefully reviewed, modified where necessary, integrated into the overall solution by me, and tested to ensure it met the assignment requirements. No other AI was used in the other parts of the assignment. The final submitted work represents my own understanding and effort. I take full responsibility for the accuracy, functionality, and academic integrity of the submitted solution.

Generative AI Tool Used: ChatGPT

Purpose of Use: Implementation and bug fixes of GPU support to speed up training

Verification Method: Manual code review

2 Exploratory Data Analysis

In this section, we analyse the given dataset `chestxray_binary_64.npz`. Through simple EDA, we identified that the images are provided as NumPy arrays of shape $(N, 64, 64)$ where N is the number of samples in the split.

The provided images have no channel axis and are in grayscale. Table 1 shows the statistics of the provided dataset. We note that the images are all 64px in size, and there is a big class imbalance. Almost $3/4$ of the training and test data are positive samples (pneumonia), which may lead to model bias and poor generalisation.

Preprint.

Table 1: Dataset split

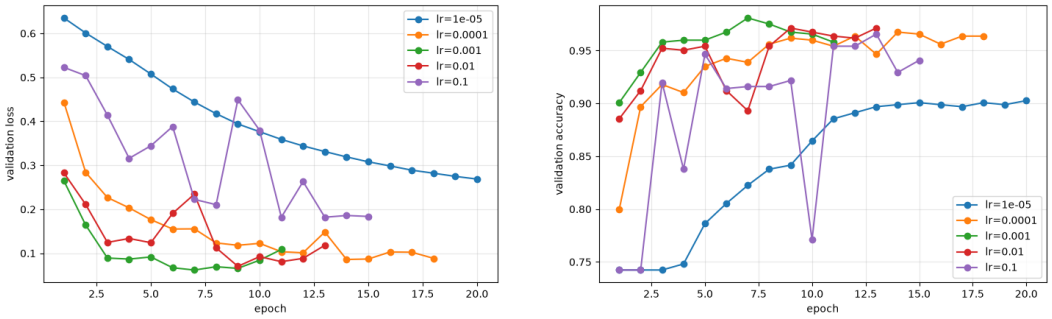
Split	Size (N, H, W)	# of positive samples
Train	$(4708, 64, 64)$	3493 (74.2%)
Test	$(624, 64, 64)$	389 (74.2%)
Validation	$(524, 64, 64)$	390 (62.5%)

2.1 Preprocessing

Following the instructions, we rescale all images to $[0, 1]$, normalise them using the mean and standard deviation of the training set, and finally added a channel axis for compatibility. The mean and standard deviation of the training set we calculated was 0.57 and 0.17 respectively.

3 Question B1

In question B1, we are tasked to train the **AlexNet-like architecture** with a fixed batch size of 128 and sweep across 5 learning rates. Figure 1 shows the plot of validation loss and accuracy against epochs for the 5 learning rates. The learning rates are colour coded in the diagram.



(a) Validation loss against epoch.

(b) Validation accuracy against epoch.

Figure 1: Validation loss and accuracy against epoch for learning rates 1e-5, 1e-4, 1e-3, 1e-2 and 1e-1. Fixed batch size of 128 is used for all experiments.

Table 2 shows the learning rate, best epoch, validation loss, validation classification error, sensitivity and specificity of the hyperparameter sweep.

Table 2: Learning rate sweep results

#	lr	best epoch	val loss	val error	sensitivity	specificity
0	1e-5	20	0.2687	0.0973	0.9614	0.7333
1	1e-4	14	0.0858	0.0324	0.9794	0.9333
2	1e-3	7	**0.0619**	**0.0191**	**0.9897**	**0.9556**
3	1e-2	9	0.0702	0.0286	0.9769	**0.9556**
4	1e-1	11	0.1818	0.0458	0.9666	0.9185

3.1 Finding the optimal learning rate

From the results, we make a few key observations: the highest learning rate 0.1 is the most unstable here, as the validation accuracy and loss fluctuates greatly with each passing epoch. The lowest learning rate 0.00001 shows the smoothest loss curve but struggles to learn meaningful features given the 20-epoch limit. The best learning rates appear to be between the two extremes, and **we select** 0.001 **as our best learning rate**, given that it reached the highest validation accuracy (ie. lowest validation classification error) at epoch 7, before hitting the early stopping criterion.

We note that apart for the highest and lowest learning rate, all other learning rates hit the early stopping criterion before the 20-epoch limit. Unsurprisingly, it follows the trend that higher learning rates often result in faster convergence (given that the best epoch decreases from 20 to 7), but setting it too high may result in instability during training and model collapse.

3.2 Sensitivity and specificity

In section 1, we identified that our dataset is highly imbalanced. During training, it is highly likely that the model may end up memorising the training data, leading to bias towards the majority class (in this case, pneumonia). Hence, we expect accuracy to be fairly high, given that the test split also mirrors this distribution. This may mislead us into thinking that we have a well-performing model, but in reality it is just biased towards the majority class.

Sensitivity and specificity are defined as:

$$\text{Sensitivity} = \frac{TP}{TP + FN} \qquad \text{Specificity} = \frac{TN}{TN + FP}$$

Sensitivity is a measure of the *true positive rate*, which evaluates how well our model can detect cases of pneumonia, given that the patient actually has pneumonia. Meanwhile, specificity is a measure of the *true negative rate*, which evaluates how well the model identifies healthy cases, given that the patient is healthy.

In a clinical setting, specificity matters a lot more compared to all the other metrics since misdiagnosing patients as healthy when they actually have pneumonia is much more serious. We use experiment **0** and our best experiment **2** as an example: while both models fare similarly in terms of sensitivity, there is **over a 20% difference in specificity**.

3.3 Final results

Evaluating the final model on the test set, Table 3 shows the final accuracy, classification error, sensitivity and specificity. From our results, we observe that the model performs very well in terms of sensitivity but falls hard in specificity, compared to the validation set.

Table 3: Final results for B1 (on test set), batch=128

lr	accuracy	classification error	sensitivity	specificity
1e-3	0.7901	0.2099	0.9974	0.4444

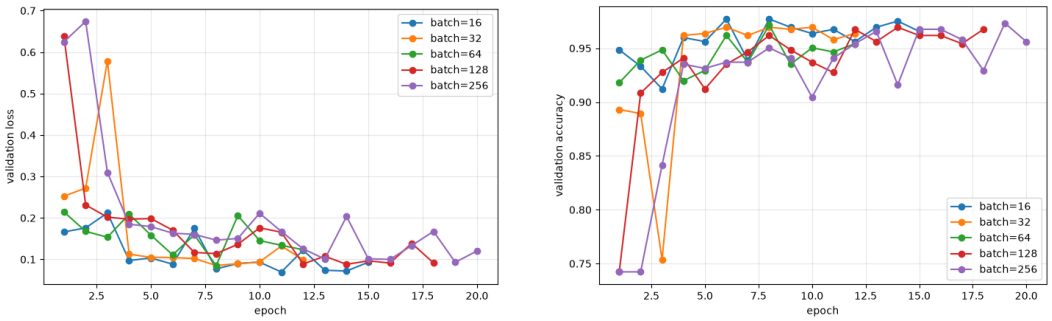
4 Question B2

In this section, we determine the optimal **batch size** for a **VGG-like architecture**, fixing the best learning rate of 0.001 found in part **B1**. We sweep across 5 different batch sizes. Figure 2 shows the plot of validation loss and accuracy against epochs for the 5 batch sizes.

Table 4 shows the batch sizes, best epoch, validation loss, validation classification error, sensitivity and specificity of the hyperparameter sweep. Best values are bolded, second-best values are underlined.

Table 4: Batch size sweep results

#	batch size	best epoch	val loss	val error	sensitivity	specificity
0	16	11	**0.0695**	0.0324	0.9769	**0.9407**
1	32	8	0.0852	0.0305	0.9871	<u>0.9185</u>
2	64	8	<u>0.0844</u>	<u>0.0286</u>	<u>0.9923</u>	0.9111
3	128	14	0.0879	0.0305	**0.9974**	0.8889
4	256	19	0.0930	**0.0267**	<u>0.9923</u>	<u>0.9185</u>



(a) Validation loss against epoch.

(b) Validation accuracy against epoch.

Figure 2: Validation loss and accuracy against epoch for batch sizes 16, 32, 64, 128 and 256. Fixed learning rate of 1e-3 is used for all experiments.

4.1 Finding the optimal batch size

From the results, we make a few key observations: as the batch size increases, we notice that validation error decreases, and the largest batch size of 256 gives the lowest validation error. However, looking at specificity, there seems to be a reverse trend - the smallest batch size actually has the highest specificity.

Since the assignment requires us to select the best batch size based on the lowest validation error, **we fix 256 as our best batch size**. However, we believe that in such a setting, it might be wiser to optimise on specificity rather than just validation accuracy - hence batch size of 16 could probably be more appropriate here.

4.2 Final results

Evaluating the final model on the test set, Table 5 shows the final accuracy, classification error, sensitivity and specificity. Compared to B1, raising the batch size from 128 to 256 resulted in better performance in the test set - the biggest gain is in specificity, which **increased from 0.44 to 0.56**.

Table 5: Final results for B2 (on test set), lr=1e-3

batch size	accuracy	classification error	sensitivity	specificity
256	0.8269	0.1731	0.9872	0.5598

5 Question B3

Next, we train a compact **ResNet-like architecture** using the best learning rate of 0.001 found in part **B1** and the best batch size 256 found in **B2**. Again, we use the same config as the previous parts, training for a maximum of 20 epochs with early stopping and patience set to 4 epochs. Evaluating the model on the test set, Table 6 shows the accuracy, classification error, sensitivity and specificity.

Table 6: Final results for B3 (on test set), lr=1e-3, batch=256

best epoch	accuracy	classification error	sensitivity	specificity
5	0.7965	0.2035	0.9974	0.4615

5.1 Comparing all three model families

We show the accuracy, sensitivity, specificity, classification error, number of learnable parameters and training FLOPs of the best performing models across all three model families that we explored in B1, B2 and B3 in Table 7.

Table 7: Architecture comparison results

arch	acc	sensitivity	specificity	error	params	FLOPs (millions)
B1 AlexNet	0.7901	**0.9974**	0.4444	0.2099	99,569	39.0636
B2 VGG	**0.8269**	0.9872	**0.5598**	**0.1731**	18,185	19.4643
B3 ResNet	0.7965	**0.9974**	0.4615	0.2035	77,169	25.2970

From the results, we observe that the performance of the models very similar based on accuracy alone. However, despite VGG requiring less training FLOPs and being a smaller model, it achieves comparable performance to the other two models. While it suffers a bit in sensitivity, it is an acceptable tradeoff given that its specificity is around 10% higher, making it more suitable for use in a clinical setting.

Nonetheless, the VGG model seems to be the most expensive to train based on the FLOPs to parameter size ratio. This means that scaling up to a larger VGG model would be more computationally expensive.

6 Question B4

In pneumonia cases, chest X-rays show abnormal white and cloudy patches. This section aims to explore if xAI (Explainable AI) methods can really identify these segments by using Grad-CAM (Gradient-weighted Class Activation Mapping). In this section, we randomly select 5 images from the test set, run predictions and overlay the heatmap generated by Grad-CAM. The results are shown in Figure 3.

6.1 Analysing positive cases with Grad-CAM

From the heatmaps, we note that AlexNet seems to provide the most representative heatmaps: it is clearly seen that there are higher activations for regions with whiter patches (which is a sign of pneumonia). Looking at VGG, the heatmaps seem to be patchy and doesn't provide much insight onto the model's decision process, as the activations are spread across the border of the image. Meanwhile, ResNet has a smaller resolution heatmap (8x8 instead of 16x16) making it appear scattered.

This is an interesting observation, given that VGG performs best but Grad-CAM seems to provide no reasonable explanation for its decision making process. Meanwhile, the gradient heatmap for AlexNet looks the most interpretable, since we can directly observe regions with white patches having greater activations, yet it performs the worst amongst all 3 models.

To end off with an open question, should we trust the model that is less accurate but more interpretable, or the model that is more accurate but not as interpretable?

7 Credits

This report was written in LaTeX using the NeurIPS 2026 style files from `https://neurips.cc`.

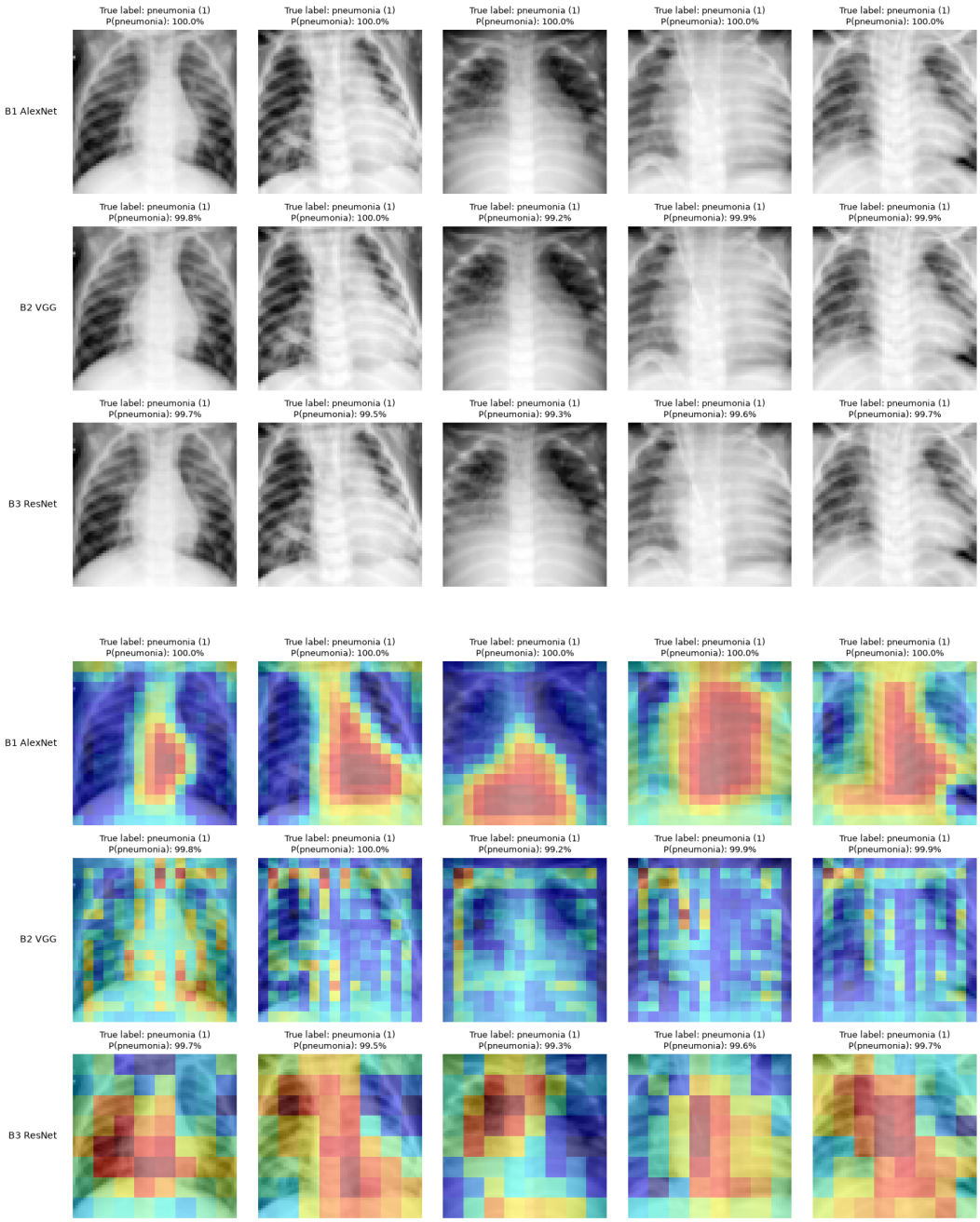


Figure 3: Comparison of 5 random samples with and without heatmaps. Model confidence scores are plotted as percentages. Red regions correspond to stronger activations. Heatmaps are computed using the outputs of the last convolutional layer of each model.